



Bringing In a Second Dataset: Weather

The Rhode Island Traffic Stops Dataset, Part Four

Records from the Stanford Open Policing Project and NOAA

Nimra Ayaz

Data Scientist & Machine Learning Engineer



RECAP

Picking up where part three left off

- The file is clean, indexed by time, and has been sliced by hour, year, and district
- This closing notebook brings in a second open dataset: daily NOAA weather readings
- The two datasets are merged by date to ask one last question about the data
- Every statistic and chart ahead is built from the published figures for both files



SECTION 1

Exploring the weather dataset

Daily weather readings for Providence, Rhode Island, from NOAA



STEP 1

Load the weather file

```
weather = pd.read_csv('weather.csv')  
weather.head()
```

- Station USW00014765, in Providence, Rhode Island
- 4,017 days on record, spanning 2005 through 2015
- 28 columns, including one for every kind of bad weather NOAA tracks

DATE	TAVG	TMIN	TMAX	AWND	WSF2
2005/01/01	44.0	35	53	8.95	25.1
2005/01/02	36.0	28	44	9.40	14.1
2005/01/03	49.0	44	53	6.93	17.0
2005/01/04	42.0	39	45	6.93	16.1
2005/01/05	36.0	28	43	7.83	17.0



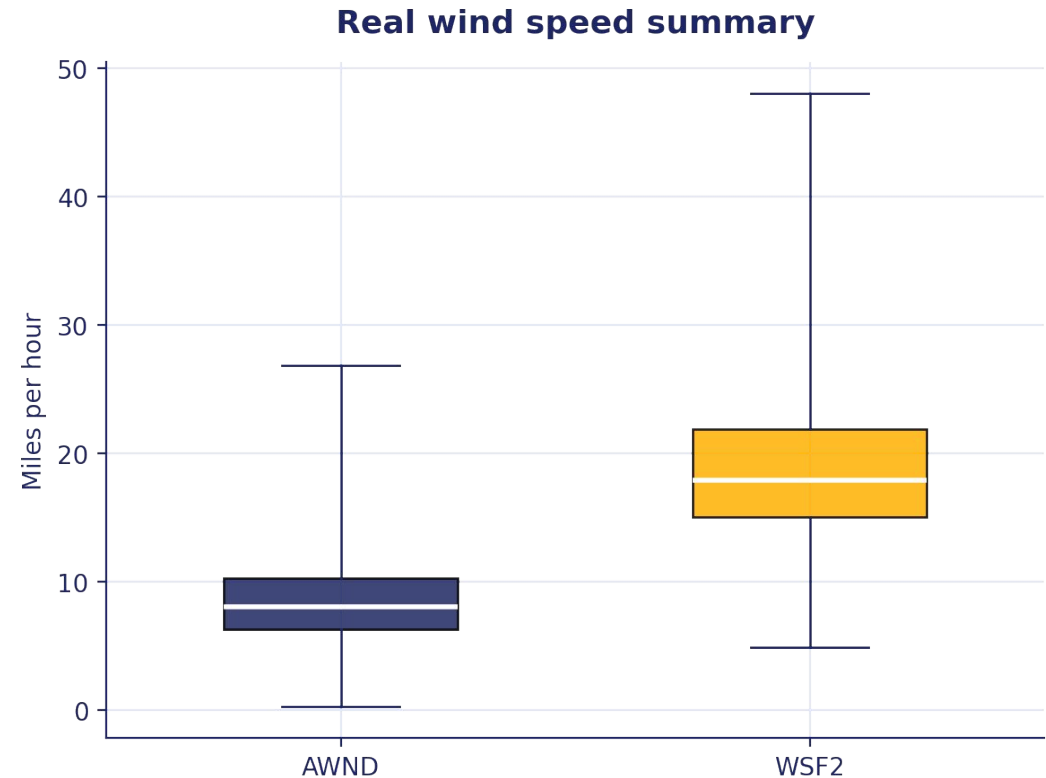
STEP 2

Examine the wind speed

```
weather[['AWND', 'WSF2']]
```

```
.describe()
```

count	4017	4017
mean	8.59	19.27
std	3.36	5.62
min	0.22	4.90
25%	6.26	15.00
50%	8.05	17.90
75%	10.29	21.90
max	26.84	48.10





Your turn: load and examine a second dataset

Open a notebook and try each step on your own data



SECTION 2

Checking whether the data is trustworthy

A few sanity checks before trusting a new dataset



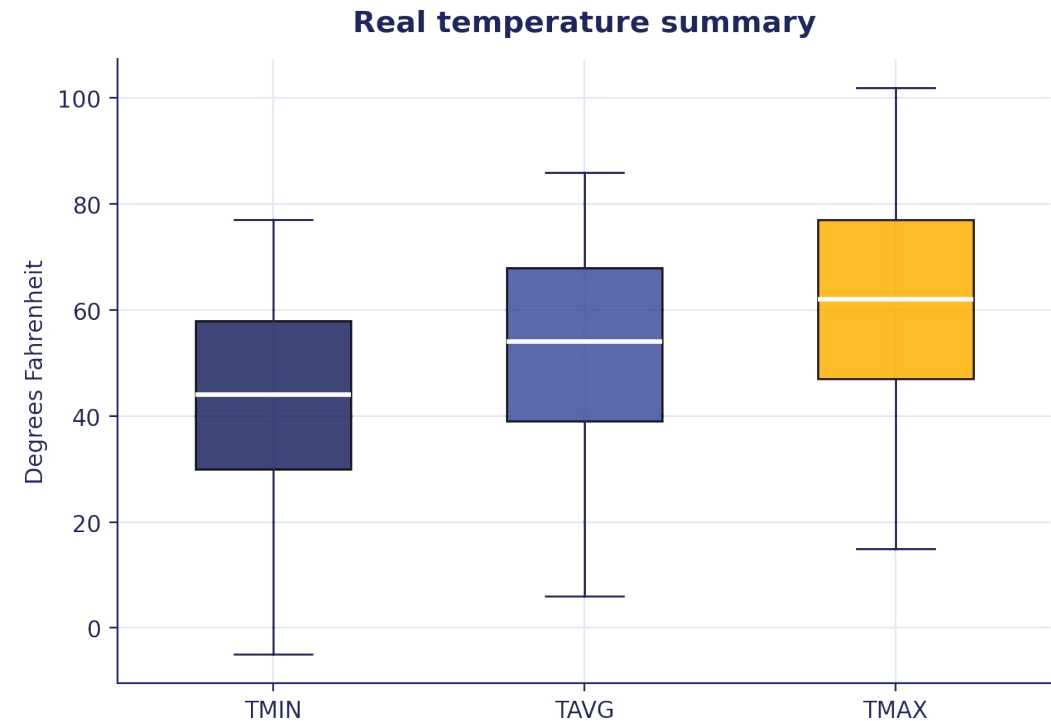
STEP 1

TMIN should never exceed TAVG or TMAX

```
weather[['TMIN','TAVG','TMAX']]
```

```
.describe()
```

TAVG has a count of only 1,217, far below TMIN and TMAX at 4,017 each. The station simply skips reporting an average some days. That is a real, normal quirk, not an error





STEP 2

TMAX minus TMIN should never be negative

```
weather['TDIFF'] = weather.TMAX - weather.TMIN  
weather.TDIFF.describe()
```



Minimum: 2

Every one of the 4,017 days has a positive spread between its high and low



Mean: 17.78

A typical Rhode Island day swings almost 18 degrees between its low and high

A negative TDIFF would flag a data problem. This file has none



Your turn: sanity check a dataset before trusting it

Open a notebook and try each step on your own data



SECTION 3

Categorizing the weather

Turning many weather columns into one simple rating

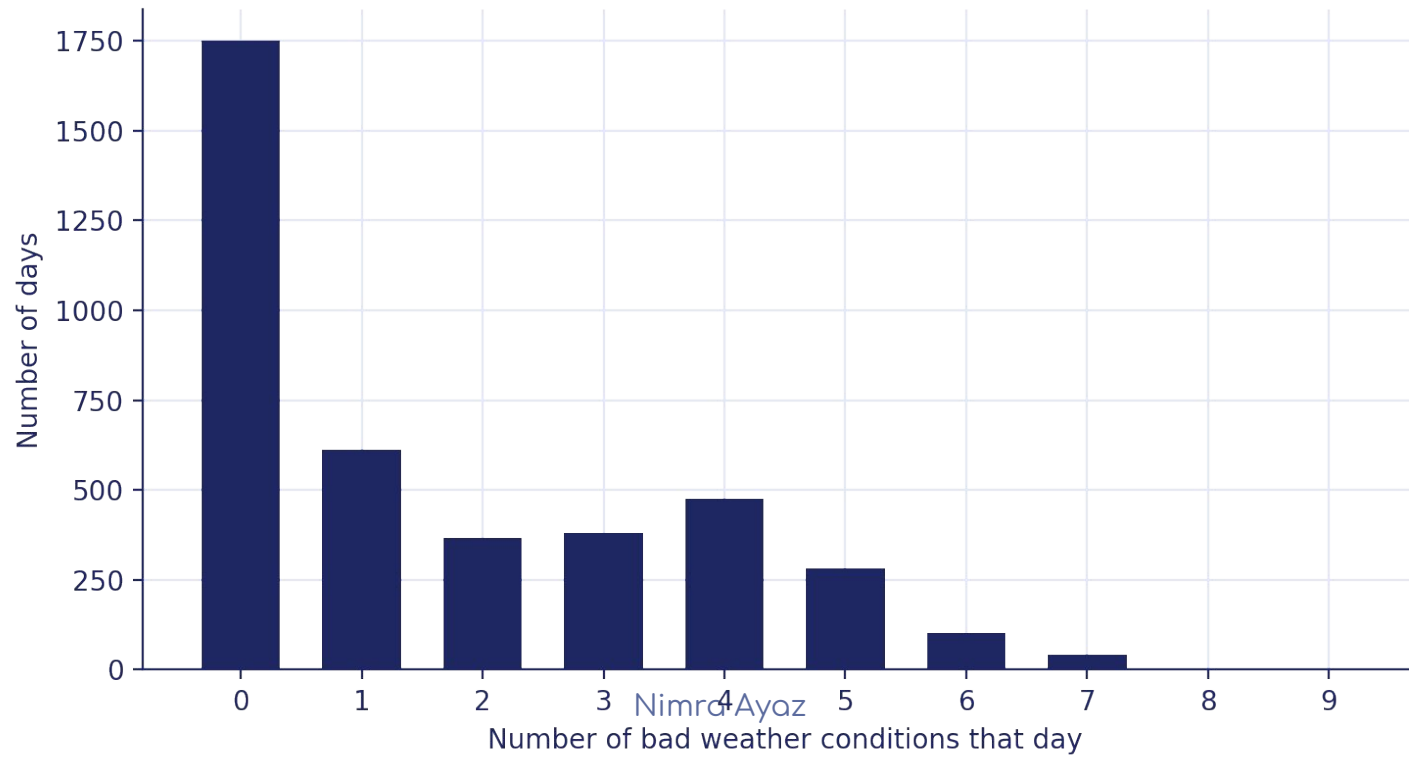


STEP 1

Count bad weather conditions per day

```
WT = weather.loc[:, 'WT01': 'WT22']  
weather['bad_conditions'] = WT.sum(axis='columns')
```

Real count of bad weather conditions per day





STEP 2

Map the count to a plain language rating

```
mapping = {0: 'good',  
           1: 'bad', 2: 'bad', ...  
           5: 'worse', ...}  
weather['rating'] = weather.bad_conditions  
                .map(mapping)
```

1,749
good days

1,836
bad days

432
worse days

*0 conditions is good, 1 to 4 is bad, 5 or more is worse,
across the 4,017 days*



STEP 3

Convert rating to an ordered category

```
cats = pd.CategoricalDtype(  
    categories=['good', 'bad', 'worse'], ordered=True)  
weather['rating'] = weather.rating.astype(cats)
```



Stores three repeated labels efficiently

A category stores each unique label once, no matter how many rows repeat it



good comes before bad comes before worse

With an order declared, comparisons like greater than behave the way you expect



Your turn: build and order a category from scratch

Open a notebook and try each step on your own data



SECTION 4

Ordered categories for stop length

The same ordered category technique, applied back to the stops file



STEP 1

Shorten the stop_duration labels

```
mapping = {'Under 15 Min': 'short',  
          '15 to 30 Min': 'medium',  
          'Over 30 Min': 'long'}  
stops['stop_length'] = stops.stop_duration.map(mapping)
```

The stop_duration column only ever holds these three text ranges across the whole file



STEP 2

Category dtype cuts memory use

```
stops.stop_length.memory_usage(deep=True)
```

8,689,481

bytes, stored as text

shrinks to

3,400,602

bytes, as an ordered category

On a small handful of rows, a category can add overhead instead of saving space. The saving shows up once a few labels repeat across many thousands of rows, exactly like here



STEP 3

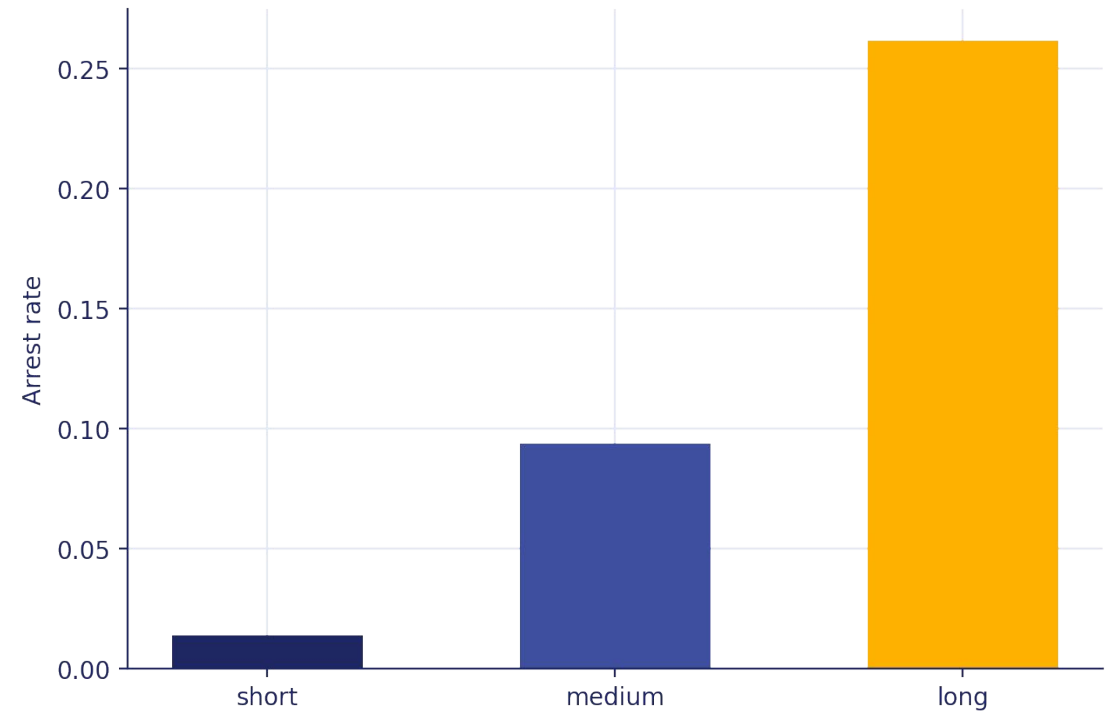
Compare categories and check the arrest rate

```
stops[stops.stop_length  
> 'short'].shape
```

16,959

of 86,536 rows run longer than short

Real arrest rate by stop length





Your turn: order, compare, and group by category

Open a notebook and try each step on your own data



SECTION 5

Merging in the weather data

A left join lines up each stop with its weather rating



STEP 1

Prepare and merge the two files

```
stops.reset_index(inplace=True)
weather_rating = weather[['DATE', 'rating']]

stops_weather = pd.merge(left=stops, right=weather_rating,
                          left_on='stop_date', right_on='DATE', how='left')
```

86,536 x 14
rows by columns before

becomes

86,536 x 16
rows by columns after



Your turn: merge two datasets on a shared date

Open a notebook and try each step on your own data



SECTION 6

Does weather affect the arrest rate

The closing question, answered with the merged data



STEP 1

Group the arrest rate by weather rating

```
stops_weather.groupby('rating')
```

```
.is_arrested.mean()
```

The rate climbs as the weather gets worse, though the difference is modest next to some of the splits seen earlier in this series

8.7%

arrest rate on good weather days

9.0%

arrest rate on bad weather days

10.7%

arrest rate on the worst weather days



STEP 2

Add violation type with a MultiIndex

```
arrest_rate = stops_weather.groupby(['violation','rating']).is_arrested.mean()
```

violation	good	bad	worse
Equipment	0.059	0.066	0.097
Moving violation	0.056	0.058	0.066
Registration/plates	0.082	0.098	0.116
Speeding	0.013	0.013	0.017

arrest_rate.loc['Moving violation', 'bad'] returns 0.058 directly. Speeding barely moves with the weather at all



STEP 3

Two roads to the same table

```
arrest_rate.unstack()
```

Starts from the already grouped Series and turns the inner index level into columns

```
stops_weather.pivot_table(  
    index='violation',  
    columns='rating',  
    values='is_arrested')
```

Starts straight from the full table instead, with no grouping step first

Both calls return the exact same table shown on the previous slide



Your turn: reshape a MultiIndex two different ways

Open a notebook and try each step on your own data

Four notebooks, one dataset



Cleaned

A messy file down to
86,536 trustworthy rows



Compared

Categories and groups
within the data



Explored

Patterns across hours,
years, and durations



Merged

A second dataset in, to
ask one final question

86,536

curated stops used in this series

509,681

total stops archived for Rhode
Island

Source: openpolicing.stanford.edu and NOAA, real and open data throughout